

Contents

Policy: least_loaded	1
Overview	1
Pipeline	1
Phase 0 — Tiering and parameter setup	1
Round 1 — Global first-choice pass	2
Main Allocation — Class A $\rightarrow$ B $\rightarrow$ C	2
Assignment rule (applied at every step)	2
Properties and trade-offs	2
When least_loaded performs best	3
Known limitations	3
Configuration	3
See also	4

Policy: least_loaded

Overview

least_loaded is the **default allocation policy**. It runs the full three-phase pipeline — Phase 0 (tiering), Round 1 (first-choice pass), and the main class-wise allocation — and at every assignment step it directs each student to the **least-loaded eligible advisor** within their preference-protection window.

The core idea is that load balance across advisors is treated as a co-equal constraint alongside preference satisfaction. No advisor is allowed to accumulate students while another eligible advisor sits empty, as long as preference and capacity constraints allow.

Pipeline

Phase 0 $\rightarrow$ Round 1 $\rightarrow$ Main Allocation (Class A $\rightarrow$ B $\rightarrow$ C)

Phase 0 — Tiering and parameter setup

Students are divided into tiers based on cohort CPI percentiles:

Tier	CPI threshold (percentile mode)	Protection window N_{tier}
A	$\geq$ 90th percentile (± 0.1 grace)	3 (or 4 if $S/F > 4$)
B	70th–90th percentile	5 (or 6 if $S/F > 4$)
C	$<$ 70th percentile	Full list

If more than 40% of students cluster in one band, the tiering switches to quartile mode (A / B1 / B2 / C). If the cohort is fewer than 10 students, all are placed in Class A with $N_{\text{tier}} = 2$.

Faculty max_load is set to $\text{floor}(S/F) + 1$ unless overridden in the input CSV.

Round 1 — Global first-choice pass

Each faculty with at least one first-choice applicant selects **exactly one student** from its applicant list. Selection defaults to the highest-CPI applicant (tie-broken by student ID), but an operator can supply manual picks. This initial pass tends to seat top-tier students at popular advisors and creates a baseline load distribution that the main allocation builds on.

Main Allocation — Class $A \rightarrow B \rightarrow C$

Unassigned students are processed class by class. Within each class:

1. **Class A** — each student is matched to the least-loaded advisor within their top N_A preferences that still has capacity. Students with no eligible advisor within N_A are promoted into Class B's pool.
 2. **Class B** (original B + promoted A) — same rule, window extends to N_B .
 3. **Class C** (everyone remaining) — no window cap; all advisors with remaining capacity are eligible. The least-loaded among them is chosen.
-

Assignment rule (applied at every step)

Given a student and their eligible candidate advisors (those within N_{tier} with remaining capacity):

1. Find the **minimum current load** among all eligible advisors.
2. If multiple advisors share that minimum load, pick the one **earliest in the student's preference list**.
3. Assign the student to that advisor and increment the load counter.

This makes load balance the **primary criterion** and preference rank the **tiebreaker**, reversing the priority order of a pure preference-first approach.

Properties and trade-offs

Property	Behaviour
Load balance	Strong — advisors are filled as evenly as possible at every step.
Preference satisfaction (PSI)	Good — most students land near the top of their list because low-load advisors often coincide with early preferences.
Merit sensitivity (NPSS)	Moderate — high-CPI students are not given explicit priority in the main allocation; their advantage comes only from Round 1 and the tier window.

Property	Behaviour
Overflow	Possible but rare — the class-wise promotion cascade ($A \rightarrow B \rightarrow C$) widens the candidate pool before resorting to the full-list Class C round; a student is only placed beyond their original N_{tier} when every advisor within every promoted pool is at capacity. Shown as a red warning badge in the UI.
Equity Retention Rate	High — spreading students by load tends to distribute different CPI tiers across advisors, preserving more of the cohort's achievable tier diversity.
MSES	Moderate — load-based routing does not explicitly target student enthusiasm; MSES is driven by the degree to which load minima align with preference ranks.
Robustness	High — performs consistently across random, clustered, and polarised cohorts.

When `least_loaded` performs best

- Cohorts with **diverse preferences** — when students do not all cluster around the same popular advisors, load-balancing and preference satisfaction align naturally.
- When **advisor equity** is a priority — departments that want to ensure no advisor is significantly more or less loaded than peers.
- **Typical or unknown cohort shapes** — because it makes no assumptions about the structure of preferences or the CPI distribution, it is the robust default.

Known limitations

- In **clustered** cohorts (many students competing for the same few advisors), load-balancing can push students to their 2nd or 3rd choice even when their 1st choice technically still has capacity, if that advisor's load is already higher than an alternative. This can feel unintuitive to high-CPI students.
- It does **not explicitly reward academic merit** beyond the tier window. Two students with very different CPIs within the same tier are treated identically.

Configuration

Set in `src/allocator/app.py`:

```
ALLOCATION_POLICY = "least_loaded"
```

Or via CLI:

```
PYTHONPATH=src python -m allocator.allocation \
  --students data/sample_students.csv \
```

```
--faculty data/sample_faculty.csv \
--policy  least_loaded
```

See also

- `adaptive_ll` — drop-in replacement that adds a structural empty-lab guarantee by iteratively widening tier caps before the main allocation; produces an identical result when the baseline caps are already sufficient.